SCHOOL MANAGEMENT SYSTEM

Standard C++17 Object-Oriented Application
University-Level Project Technical Documentation & Viva Handbook

---

Course: Object-Oriented Programming & Data Structures

Language: Standard C++ (ISO/IEC 14882:2017 - C++17)

Compiler: MinGW-w64 / GCC 16.1.0

Architecture: 3-Tier Layered Architecture (Model, Manager, View)

Storage: Flat File Handling (.txt) with Delimited Data Serialization

Academic Level: Undergraduate Computer Science / Software Engineering

Date: Academic Year 2026-2027

Chapter 2: Table of Contents

Ch.	Chapter Title	Core Content Overview
1	Cover Page	Title, Course metadata, Standards, and Academic Information
2	Table of Contents	Navigational outline of all 27 chapters
3	Project Introduction	Background, Motivation, Problem Statement, and Scope
4	Project Objectives	Core academic goals, functional targets, and boundaries
5	Technologies and Tools Used	C++17, MinGW GCC, Standard Template Library (STL), Win32 API
6	Complete Project Folder Structure	Directory tree, responsibilities, and file locations
7	Explanation of Every File	In-depth examination of all .h, .cpp, .bat, and .txt files
8	Explanation of Every Library Used	STL headers (vector, string, iomanip, filesystem, etc.) & Win32
9	C++ Fundamentals Used in the Project	Streams, preprocessor directives, pointers, references, control structures
10	Object-Oriented Programming Concepts	Encapsulation, abstraction, classes, composition, access modifiers
11	Complete Explanation of Every Class	Student, StudentManager, AttendanceRecord, AttendanceManager, etc.
12	Complete Explanation of Every Function	Line-by-line review of every member function, signature, and logic
13	Student Information Management Module	CRUD operations, duplicate prevention, and display tables
14	Attendance Management Module	Attendance logging, same-day duplicate check, percentage formulas
15	Examination Results Management Module	Marks recording, validation, dynamic percentage, and letter grading
16	Student Profile Card	All-in-one console dashboard layout and cross-module aggregation
17	Student Report Card	Official academic transcript formatting, subject tables, PASS/FAIL
18	Class Merit List	Sorting algorithms, ranking calculation, and equal-percentage tie handling
19	File Handling and Data Storage	Serialization, ifstream/ofstream, startup auto-load, cascading delete
20	Complete Program Execution Flow	Step-by-step lifecycle from bootstrap to clean termination
21	Algorithms and Calculations	Percentage, grade thresholds, dense vs competition ranking, sanitization
22	Input Validation and Error Handling	Delimiter shielding, type safety, range enforcement, date checks
23	Compilation and Execution Instructions	CLI g++ command, batch scripts, compiler flags (-Wall -Wextra)
24	Testing and Sample Outputs	Console transcript captures, screen views, edge case results
25	Project Limitations	Disk I/O scale, linear searches, single-threaded console scope
26	Conclusion	Project summary, learning outcomes, and future academic extensions
27	University Viva Questions and Answers	70 Comprehensive Questions (50 Basic/Intermediate + 20 Advanced)

Chapter 3: Project Introduction

The **School Management System** is an academic-grade console software application engineered in modern standard C++ (ISO/IEC 14882:2017). Designed for educational administration, it provides a centralized system to coordinate the three most critical operational pillars of a school: **Student Information Management**, **Student Attendance Management**, and **Examination Results Management**.

In many institutions, student tracking suffers from fragmented record-keeping, manual data discrepancies, and reliance on bulky external database engines that obscure underlying computer science mechanics. This project implements a lightweight, zero-dependency, and strictly modular solution that demonstrates the power of clean Object-Oriented Programming (OOP) and persistent file input/output without requiring heavyweight frameworks such as Qt, Electron, or SQL relational database servers.

3.1 Problem Statement

Academic record systems face several common points of failure:

- **Data Redundancy and Duplication:** Multiple students assigned conflicting registration identifiers, or multiple attendance marks submitted for the same calendar date.
- **Data Inconsistency (Orphan Records):** When a student record is removed, unlinked historical records (such as examination marks or attendance entries) frequently persist in disconnected tables.
- **Input Vulnerability:** Standard console applications frequently crash or enter infinite loops when non-numeric characters are typed into numeric prompts (a classic `std::cin` stream failure).
- **Data Fragility:** Lack of automated persistence means data is lost immediately once power or the terminal session terminates.

3.2 Architectural Solution

This project solves these fundamental problems using a disciplined, decoupled 3-tier console architecture:

> **The Three Architectural Layers**
>
> 1. **Entity Models (Core Objects):** Pure encapsulated domain data classes (`Student`, `AttendanceRecord`, `ExamRecord`) maintaining individual state and serialization formats.
> 2. **Managers (Business Logic & File I/O):** Independent operational managers (`StudentManager`, `AttendanceManager`, `ExaminationManager`) holding in-memory collections (`std::vector`) and managing persistent I/O with flat `.txt` files.
> 3. **Console Presentation & UI Utilities:** ANSI-driven virtual terminal rendering, boundary-safe string formatting, and a hierarchical menu system coordinated in `main.cpp`.

Chapter 4: Project Objectives

The primary goal of this project is to deliver a functional, resilient, and academically rigorous management system that strictly satisfies university software engineering standards.

4.1 Core Functional Objectives

- **Student Information Lifecycle:** Enable full CRUD (Create, Read, Update, Delete) capability for student profiles, enforcing unique Student IDs, valid age boundaries (4 to 100 years), and class assignments.
- **Attendance Verification:** Accurately log attendance status (`Present` or `Absent`) indexed by Student ID and date (`YYYY-MM-DD`). Prevent duplicate records for any student on the same date.
- **Academic Evaluation & Grading:** Record subject-wise examination scores, enforce marks validity ($0 \le ext{Obtained} \le ext{Total}$), calculate percentages dynamically, and automatically compute letter grades ($A+, A, B, C, D, F$).
- **Relational Data Synchronization:** Enforce data integrity such that when a student is deleted, all historical attendance and examination records belonging to that student are cascaded and permanently deleted across all data files.

4.2 Non-Functional & Technical Objectives

- **Zero External Dependencies:** Use only standard ISO C++17 libraries. Avoid Qt, external GUI frameworks, or SQL databases.
- **Robust Input Sanitization:** Eliminate stream crashing caused by erroneous terminal input. Every input prompt must handle empty inputs, malformed characters, and buffer overflows safely.
- **Human-Readable Persistent Storage:** Use delimited plain text files (`data/students.txt`, `data/attendance.txt`, `data/results.txt`) that can be inspected, backed up, and edited with any standard text editor.
- **Professional Console UI/UX:** Implement box-drawing UTF-8 borders, ANSI color highlighting, and tabular alignment using `std::setw` and `std::left` / `std::right` manipulators.

Chapter 5: Technologies and Tools Used

5.1 Programming Language & Standard

- **Language:** Standard C++ (ISO/IEC 14882:2017, commonly known as **C++17**).
- **Standard Features Utilized:**
  - `<filesystem>`: Used for path manipulation and recursive directory creation (`std::filesystem::create_directories`) to guarantee storage existence.
  - **Structured Binding and Lambdas:** Used in predicates for `std::remove_if` and table formatting.
  - **Stream State Handling:** Modern safe parsing with `std::stoi` and `std::stod` with exact string end-index verification.

5.2 Compiler Toolchain

- **Compiler:** MinGW-w64 port of the GNU Compiler Collection (**GCC 16.1.0** / MSYS2 Project).
- **Compilation Flags:**
  - `-std=c++17`: Mandates strict C++17 ISO compliance.
  - `-Wall`: Enables all standard compiler warnings regarding questionable code constructs.
  - `-Wextra`: Enables supplementary warnings for enhanced code safety and quality assurance.

5.3 Platform Integration (Win32 API)

To ensure modern ANSI escape sequences and UTF-8 box characters render flawlessly across Command Prompt, Windows PowerShell, and Windows Terminal, the application interfaces directly with the Windows Console Subsystem via `<windows.h>` wrapped inside conditional compilation preprocessor guards (`#ifdef _WIN32`):

```
#ifdef _WIN32
#include <windows.h>
#endif

inline void initConsole() {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);
    if (hOut != INVALID_HANDLE_VALUE) {
        DWORD dwMode = 0;
        if (GetConsoleMode(hOut, &dwMode)) {
            dwMode |= ENABLE_VIRTUAL_TERMINAL_PROCESSING;
            SetConsoleMode(hOut, dwMode);
        }
    }
#endif
}
```

Chapter 6: Complete Project Folder Structure

The project adheres strictly to the prescribed university workspace organization:

```
School-Management-System/
│
├── main.cpp                  # Main driver, orchestrator, and Student Profile Card view
├── student.h                 # Declarations: Student, StudentManager, and UI Utilities
├── student.cpp               # Implementation: Student entity and StudentManager business logic
├── attendance.h              # Declarations: AttendanceRecord and AttendanceManager
├── attendance.cpp            # Implementation: AttendanceRecord and AttendanceManager business logic
├── examination.h             # Declarations: ExamRecord, StudentAcademicSummary, ExaminationManager
├── examination.cpp           # Implementation: ExamRecord, ExaminationManager, Report Card, Merit List
├── compile.bat               # Windows batch script for automated compilation
├── run.bat                   # Windows batch script for executable execution
├── SchoolManagementSystem.exe # Compiled 64-bit binary executable
│
└── data/                     # Persistent storage directory
    ├── students.txt          # Serialized student records
    ├── attendance.txt        # Serialized attendance logs
    └── results.txt           # Serialized subject examination marks
```

Directory / Component	Purpose & Responsibility
`Root Directory (School-Management-System/)`	Houses all translation units (.cpp), interface headers (.h), build scripts (.bat), and the compiled executable binary.
`data/ Subfolder`	Dedicated storage directory isolated from source code, holding the three flat text database files. Automatically created by code if missing.

Chapter 7: Explanation of Every File

7.1 Header Files (.h)

1. **`student.h`:** Contains the declaration of the `Student` entity, the `StudentManager` class, and the standalone `UI` namespace. It introduces all common terminal colors, box drawing routines, and input parsing functions (such as `getIntInput`, `getStringInput`, and `getDateInput`).
2. **`attendance.h`:** Defines the `AttendanceRecord` class (representing an attendance event with Student ID, Date, and Status) and the `AttendanceManager` class, which manages collection queries, duplicate prevention, and percentage formulas.
3. **`examination.h`:** Defines the `ExamRecord` class (representing subject scores), the `StudentAcademicSummary` data transfer struct, and the `ExaminationManager` class, which handles mark calculation, grading rules, report card layout, and ranking.

7.2 Source Files (.cpp)

1. **`main.cpp`:** The core driver file. Contains the program entry point `main()`, coordinates instances of the three manager classes, handles main menu navigation, and renders the **Student Profile Card**.
2. **`student.cpp`:** Implements constructors, getters, setters, file serialization (`toFileString`, `fromFileString`), file loading, file saving, duplicate checks, student updates, and student deletions.
3. **`attendance.cpp`:** Implements attendance logging, file I/O for `attendance.txt`, same-day verification, historical query filtering by student ID, and attendance percentage evaluation.
4. **`examination.cpp`:** Implements mark persistence for `results.txt`, range validation, automatic letter grade evaluation, the **Academic Report Card** layout, and the **Class Merit List** sorting algorithm.

7.3 Build Scripts & Storage Files

1. **`compile.bat`:** A Windows shell script that invokes `g++ -std=c++17 -Wall -Wextra main.cpp student.cpp attendance.cpp examination.cpp -o SchoolManagementSystem.exe` and verifies exit codes.
2. **`run.bat`:** Checks if the binary exists; if not, triggers `compile.bat` before executing the application.
3. **`data/students.txt`:** Plaintext file storing student records formatted as `ID|Name|Age|Class`.
4. **`data/attendance.txt`:** Plaintext file storing attendance records formatted as `StudentID|Date|Status`.
5. **`data/results.txt`:** Plaintext file storing academic marks formatted as `StudentID|Subject|ObtainedMarks|TotalMarks`.

Chapter 8: Explanation of Every Library Used

The application utilizes strictly standard C++ library headers. Each library serves a specific, well-defined role:

Header Library	Purpose & Role in Project	Key Functions / Classes Used
`<iostream>`	Standard console stream I/O.	`std::cin`, `std::cout`, `std::endl`
`<string>`	Dynamic string manipulation and memory management.	`std::string`, `length()`, `substr()`, `find()`, `to_string()`
`<vector>`	Dynamic array container for in-memory records.	`std::vector`, `push_back()`, `emplace_back()`, `erase()`, `size()`
`<iomanip>`	Console output formatting and tabular column alignment.	`std::setw()`, `std::left`, `std::right`, `std::setprecision()`, `std::fixed`
`<fstream>`	File stream operations for reading and writing data files.	`std::ifstream`, `std::ofstream`, `is_open()`, `close()`, `std::ios::trunc`
`<sstream>`	In-memory string stream tokenization of delimited records.	`std::stringstream`, `std::getline(ss, token, '|')`
`<algorithm>`	Standard sorting, searching, and collection filtering algorithms.	`std::sort()`, `std::find()`, `std::remove_if()`, `std::min()`
`<cctype>`	Character classification and case transformation.	`std::isdigit()`, `std::toupper()`
`<cmath>`	Floating-point mathematical comparisons and absolute values.	`std::abs()` (used for floating-point percentage tie comparisons)
`<filesystem>`	Filesystem inspection and safe folder generation (C++17).	`std::filesystem::path`, `std::filesystem::create_directories`
`<windows.h>`	Win32 console API for UTF-8 and ANSI escape code enabling.	`SetConsoleOutputCP()`, `GetStdHandle()`, `SetConsoleMode()`

Chapter 9: C++ Fundamentals Used in the Project

9.1 Preprocessor Directives and Header Guards

Every header file uses conditional inclusion guards (`#ifndef`, `#define`, `#endif`) to prevent duplicate definition errors during multi-file compilation:

```
#ifndef STUDENT_H
#define STUDENT_H
// Declarations...
#endif // STUDENT_H
```

9.2 Pointers vs References

- **Pass-by-Reference-to-Const (`const Type&`):** Used extensively in getters, search parameters, and display routines (e.g., `const StudentManager& studentMgr`) to eliminate expensive memory copies of large objects while enforcing immutability.
- **Raw Pointers as Optional References:** `StudentManager::findStudent(const std::string& id)` returns `const Student*` (or non-const `Student*`). This idiom cleanly communicates success (a valid memory pointer) or failure (`nullptr`) without throwing exceptions.

9.3 Robust String Tokenization with String Streams

Record deserialization splits fields using `std::stringstream` and the 3-argument `std::getline()` with a custom delimiter character (`'|'`):

```
std::stringstream ss(line);
std::string id, name, ageStr, studentClass;
if (std::getline(ss, id, '|') &&
    std::getline(ss, name, '|') &&
    std::getline(ss, ageStr, '|') &&
    std::getline(ss, studentClass)) {
    int age = std::stoi(ageStr);
    return Student(id, name, age, studentClass);
}
```

Chapter 10: Object-Oriented Programming Concepts

10.1 Encapsulation

All entity classes maintain strictly `private` member data. External code cannot modify an attribute directly; it must invoke validated public mutators (setters).

Code Verification: Student Class Encapsulation

```
class Student {
private:
    std::string id;
    std::string name;
    int age;
    std::string studentClass;
public:
    std::string getId() const;
    void setName(const std::string& name);
    // ...
};
```

10.2 Constructors and Destructors

- **Default Constructors:** Provide safe zero-initialization (e.g., `age(0)`, `obtainedMarks(0.0)`) required when creating blank instances for stream reading.
- **Parameterized Constructors:** Construct fully-formed, valid entities in a single atomic statement using member initializer lists:

```
Student::Student(const std::string& id, const std::string& name, int age, const std::string& studen
    : id(id), name(name), age(age), studentClass(studentClass) {}
```

10.3 Class Relationships (Composition, Aggregation, and Association)

- **Composition / Strong Ownership:** `StudentManager` owns a collection of `Student` objects (`std::vector<Student> students`). When the manager instance is destroyed, all student objects it contains are deallocated from memory.
- **Association / Dependency:** `AttendanceManager` and `ExaminationManager` do not inherit from or own `StudentManager`. Instead, they accept a temporary reference parameter (`const StudentManager& studentMgr`) to verify student existence during marks entry or attendance logging.

10.4 OOP Concepts Not Implemented (Design Justification)

In academic evaluation, it is critical to explicitly explain why certain OOP features were *deliberately omitted*:

- **Inheritance (`class Dog : public Animal`):** Not used because a Student, an Attendance Record, and an Exam Record do not share an "is-a" relationship. Forcing an inheritance tree between unrelated entities is an anti-pattern.
- **Polymorphism / Virtual Tables (`virtual`):** Not required because entity collections are homogeneous (all students are of type `Student`; there are no polymorphic sub-types like `GraduateStudent` or `PartTimeStudent`). Omitting

`virtual` eliminates vtable pointer overhead and simplifies memory layout.

Chapter 11: Complete Explanation of Every Class

11.1 Class: `Student`

Location: `student.h` (lines 208-233), implemented in `student.cpp` (lines 11-45).
Purpose: Acts as the fundamental entity model representing a registered student within the educational institution.

Member Attribute	Data Type	Access	Description
`id`	`std::string`	`private`	Unique institutional identifier (e.g., "S101"). Immutable after creation.
`name`	`std::string`	`private`	Full name of the student (e.g., "Alice Johnson").
`age`	`int`	`private`	Age in complete years (bounded between 4 and 100).
`studentClass`	`std::string`	`private`	Grade, section, or cohort code (e.g., "CS-101").

11.2 Class: `StudentManager`

Location: `student.h` (lines 238-266), implemented in `student.cpp` (lines 50-312).
Purpose: Serves as the repository and controller for all student data, coordinating memory cache (`students` vector), flat file synchronization (`students.txt`), duplicate checks, and user interaction.

Member Attribute	Data Type	Access	Description
`students`	`std::vector<Student>`	`private`	Dynamic in-memory storage array of all student instances.
`filePath`	`std::string`	`private`	Relative file system path to the storage file ("data/students.txt").

11.3 Class: `AttendanceRecord`

Location: `attendance.h` (lines 13-34), implemented in `attendance.cpp` (lines 11-33).
Purpose: Encapsulates a single attendance observation for a specific student on a specific date.

Member Attribute	Data Type	Access	Description
`studentId`	`std::string`	`private`	Foreign key pointing to the unique ID of an existing student.
`date`	`std::string`	`private`	Observation date formatted in strict ISO 8601 (`YYYY-MM-DD`).
`status`	`std::string`	`private`	Normalized attendance state: either "Present" or "Absent".

11.4 Class: `AttendanceManager`

Location: `attendance.h` (lines 39-65), implemented in `attendance.cpp` (lines 38-238).
Purpose: Manages attendance persistence, duplicate check validation, cross-module student verification, and percentage calculations.

11.5 Class: `ExamRecord`

Location: `examination.h` (lines 13-37), implemented in `examination.cpp` (lines 12-52).
Purpose: Represents academic performance in an individual subject.

Member Attribute	Data Type	Access	Description
`studentId`	`std::string`	`private`	Foreign key matching a valid Student ID.
`subject`	`std::string`	`private`	Academic subject name (e.g., "Programming Fundamentals").
`obtainedMarks`	`double`	`private`	Numerical marks earned by the student ($0 \le ext{obtained} \le ext{total}$).
`totalMarks`	`double`	`private`	Maximum possible score for the examination (must be > 0).

11.6 Struct: `StudentAcademicSummary`

Location: `examination.h` (lines 42-52).
Purpose: A Data Transfer Object (DTO) aggregating a student's full examination record for report card and merit list generation. Contains `studentId`, `name`, `studentClass`, `totalObtained`, `totalMax`, `percentage`, `overallGrade`, `hasFailedSubject`, `subjectCount`, and `rank`.

11.7 Class: `ExaminationManager`

Location: `examination.h` (lines 57-84), implemented in `examination.cpp` (lines 56-528).
Purpose: Manages marks persistence (`results.txt`), letter grade evaluation, report card transcript generation, and merit list calculations.

Chapter 12: Complete Explanation of Every Function

This chapter examines every function implemented across the system, detailing its signature, parameters, execution logic, and code implementation.

12.1 Utility & Console Input Functions (namespace `UI`)

1. `UI::initConsole()`

- **Return Type:** `void` | **Parameters:** None
- **Purpose:** Configures Windows console subsystems to render UTF-8 box characters and interpret ANSI virtual terminal color codes.
- **Logic:** Executes `SetConsoleOutputCP(CP_UTF8)` and retrieves the standard output handle via `GetStdHandle(STD_OUTPUT_HANDLE)`. Retrieves the current console mode bitmask with `GetConsoleMode`, bitwise-ORs it with `ENABLE_VIRTUAL_TERMINAL_PROCESSING`, and updates it via `SetConsoleMode`.

2. `UI::getStringInput(const std::string& prompt, bool allowEmpty)`

- **Return Type:** `std::string` | **Parameters:** `prompt` (string), `allowEmpty` (boolean flag)
- **Purpose:** Reads an entire line of text safely, trims whitespace, and rejects inputs containing the reserved file delimiter character `'|'`.
- **Logic:** Loops until a valid input is received. Uses `std::getline(std::cin, input)`. Trims leading and trailing spaces. If `!allowEmpty && input.empty()`, prints an error message. If `input.find('|') != std::string::npos`, warns the user that `'|'` is reserved and prompts again.

3. `UI::getIntInput(const std::string& prompt, int minVal, int maxVal)`

- **Return Type:** `int` | **Parameters:** `prompt` (string), `minVal` (minimum boundary), `maxVal` (maximum boundary)
- **Purpose:** Guarantees crash-proof integer input within numerical boundaries.
- **Logic:** Reads a full string with `std::getline` to prevent stream lock. Calls `std::stoi(line, &idx)` inside a `try-catch` block. Verifies that `idx == line.length()` (rejecting mixed inputs like "12abc"). Ensures `val >= minVal && val <= maxVal`.

4. `UI::getDoubleInput(const std::string& prompt, double minVal, double maxVal)`

- **Return Type:** `double` | **Parameters:** `prompt`, `minVal`, `maxVal`
- **Purpose:** Reads validated floating-point marks and scores safely.
- **Logic:** Uses `std::stod(line, &idx)` inside a `try-catch` block with boundary enforcement ($minVal \le val \le maxVal$).

5. `UI::getDateInput(const std::string& prompt)`

- **Return Type:** `std::string` | **Parameters:** `prompt`
- **Purpose:** Validates calendar date input in strict `YYYY-MM-DD` format.
- **Logic:** Verifies that the string length is exactly 10 characters with hyphens at index 4 and 7. Checks that all other 8 characters are numeric via `std::isdigit`. Parses substring integers for Year (2000-2099), Month (1-12), and Day (1-31).

12.2 `Student` and `StudentManager` Functions

6. `Student::toFileString() const`

```
std::string Student::toFileString() const {
    return id + "|" + name + "|" + std::to_string(age) + "|" + studentClass;
}
```

Logic: Serializes the student entity into a single delimited line suitable for storage in `students.txt`.

7. `Student::fromFileString(const std::string& line)`

```
Student Student::fromFileString(const std::string& line) {
    std::stringstream ss(line);
    std::string id, name, ageStr, studentClass;
    if (std::getline(ss, id, '|') &&
        std::getline(ss, name, '|') &&
        std::getline(ss, ageStr, '|') &&
        std::getline(ss, studentClass)) {
        try {
            int age = std::stoi(ageStr);
            return Student(id, name, age, studentClass);
        } catch (...) {}
    }
    return Student();
}
```

Logic: Deserializes a text line using `std::stringstream` and custom delimiter extraction. Returns an instantiated `Student` object or an empty fallback instance if parsing fails.

8. `StudentManager::loadFromFile()`

Logic: Clears the current in-memory vector. Opens `std::ifstream` on `filePath`. If the file does not exist, returns `true` safely (clean first startup). Reads each line, skips empty rows, converts via `Student::fromFileString()`, and appends valid instances to `students`.

9. `StudentManager::saveToFile() const`

Logic: Extracts the parent directory path using `std::filesystem::path` and calls `std::filesystem::create_directories()` to ensure the folder exists. Opens `std::ofstream` with `std::ios::trunc` and writes each serialized student followed by a newline.

10. `StudentManager::addStudent(const Student& student)`

Logic: Checks `studentExists(student.getId())`. If duplicate, rejects with `false`. Otherwise, executes `students.push_back(student)`, calls `saveToFile()`, and returns `true`.

11. `StudentManager::deleteStudent(const std::string& id)`

```
bool StudentManager::deleteStudent(const std::string& id) {
    auto it = std::remove_if(students.begin(), students.end(),
        [&id](const Student& s) { return s.getId() == id; });
    if (it != students.end()) {
        students.erase(it, students.end());
        saveToFile();
        return true;
    }
    return false;
}
```

Logic: Implements the C++ erase-remove idiom. Evaluates lambda predicate `s.getId() == id`, shifts remaining elements, erases the removed range, updates disk storage via `saveToFile()`, and returns `true`.

12.3 `AttendanceManager` Functions

12. `AttendanceManager::hasAttendance(const std::string& studentId, const std::string& date) const`

Logic: Performs a linear scan through `records` to detect if a record with identical `studentId` and `date` already exists. Prevents multiple attendance submissions on the same calendar day.

13. `AttendanceManager::calculateAttendancePercentage(const std::string& studentId, int& presentCount, int& totalDays) const`

```
double AttendanceManager::calculateAttendancePercentage(const std::string& studentId, int& presentCount
    presentCount = 0;
    totalDays = 0;
    for (const auto& rec : records) {
        if (rec.getStudentId() == studentId) {
            totalDays++;
            if (rec.getStatus() == "Present") {
                presentCount++;
            }
        }
    }
    if (totalDays == 0) return 0.0;
    return (static_cast<double>(presentCount) / totalDays) * 100.0;
}
```

Logic: Calculates attendance percentage using integer references for present days and total days. Protects against division-by-zero when no sessions have been logged.

14. `AttendanceManager::deleteRecordsForStudent(const std::string& studentId)`

Logic: Removes all attendance entries matching `studentId` using `std::remove_if` and commits the updated vector to `attendance.txt`. Invoked during cascading student deletion.

12.4 `ExaminationManager` Functions

15. `ExaminationManager::calculateGrade(double percentage)`

```
std::string ExaminationManager::calculateGrade(double percentage) {
    if (percentage >= 90.0) return "A+";
    if (percentage >= 80.0) return "A";
    if (percentage >= 70.0) return "B";
    if (percentage >= 60.0) return "C";
    if (percentage >= 50.0) return "D";
    return "F";
}
```

Logic: Standard university letter grade decision structure. Evaluates boundaries in descending order.

16. `ExaminationManager::addOrUpdateRecord(...)`

Logic: Searches for an existing entry matching both `studentId` and `subject`. If found, updates the obtained and total marks in place. If not found, appends a new `ExamRecord`. Saves changes immediately to `results.txt`.

Chapter 13: Student Information Management Module

Module 1 coordinates all operations relating to student profiles. Accessible via Option `1` on the Main Menu, it provides an isolated sub-menu:

```
========================================================================
               MODULE 1: STUDENT INFORMATION MANAGEMENT
========================================================================
  [1] Add New Student
  [2] Display All Students
  [3] Search Student by ID
  [4] Update Student Information
  [5] Delete Student Record
  [6] View Student Profile Card (All-in-One Summary)
  [0] Back to Main Menu
```

13.1 Adding a New Student

The user is prompted for Student ID, Full Name, Age, and Class. The system executes `studentExists(id)` to ensure uniqueness. If the ID exists, an error message is displayed and the operation halts without modifying storage.

13.2 Displaying Enrolled Students

Renders all active student records in an aligned tabular grid using `std::setw` manipulators:

```
+------------+---------------------------------+-----+-----------------+
| Student ID | Full Name                       | Age | Class           |
+------------+---------------------------------+-----+-----------------+
| S101       | Alice Johnson                   |  19 | CS-101          |
| S102       | Bob Smith                       |  20 | CS-101          |
+------------+---------------------------------+-----+-----------------+
```

13.3 Cascading Deletion Synchronization

Deleting a student triggers cascading data deletion in `main.cpp`:

```
if (studentMgr.handleDeleteStudent(deletedId)) {
    attendanceMgr.deleteRecordsForStudent(deletedId);
    examMgr.deleteRecordsForStudent(deletedId);
    UI::printInfo("Synchronized: All associated records removed.");
}
```

This eliminates orphaned records across `attendance.txt` and `results.txt`.

Chapter 14: Attendance Management Module

Module 2 manages attendance tracking and analytics. Key features include:

- **Student Existence Check:** Verifies the Student ID against `StudentManager` before prompting for attendance details.
- **Date Validation:** Enforces the `YYYY-MM-DD` calendar format.
- **Duplicate Prevention:** Prevents multiple attendance records for the same student on the same date via `hasAttendance(id, date)`.
- **Attendance Analytics:** Computes the percentage of present days and displays a warning if attendance falls below the 75% institutional requirement.

Chapter 15: Examination Results Management Module

Module 3 manages academic examination marks and grading rules:

- **Boundary Validation:** Ensures Total Marks > 0 and $0 \le ext{Obtained Marks} \le ext{Total Marks}$.
- **Subject Overwrite Protection:** If marks for a subject have already been entered for a student, prompts the user for confirmation before updating the existing entry.
- **Automated Grading:** Applies standardized university grade boundaries ($A+$ to F).

Chapter 16: Student Profile Card

The **Student Profile Card** aggregates data across all three subsystems into a unified console dashboard.

Profile Card Cross-Module Data Integration

- **Personal Details:** Student ID, Full Name, Age, and Class from `StudentManager`.
- **Attendance Summary:** Total Sessions, Present Days, Absent Days, Percentage, and Status (Regular vs Shortage) from `AttendanceManager`.
- **Academic Performance:** Subject Count, Cumulative Marks, Percentage, Overall Grade, and Standing from `ExaminationManager`.

```
╔═══════════════════════════════════════════════════════════════╗
║                    OFFICIAL STUDENT PROFILE CARD              ║
║                     SCHOOL MANAGEMENT SYSTEM                  ║
╠═══════════════════════════════════════════════════════════════╣
║ [1] PERSONAL DETAILS                                          ║
║   Student ID : S101                Class : CS-101             ║
║   Full Name  : Alice Johnson       Age   : 19 years           ║
╠═══════════════════════════════════════════════════════════════╣
║ [2] ATTENDANCE SUMMARY                                        ║
║   Total Days : 2                   Present Days : 2           ║
║   Absent Days: 0                   Attendance % : 100.0%      ║
║   Status     : REGULAR / ELIGIBLE (>= 75%)                    ║
╠═══════════════════════════════════════════════════════════════╣
║ [3] EXAMINATION SUMMARY                                       ║
║   Subjects   : 2                   Total Marks  : 186 / 200   ║
║   Percentage : 93.00%              Overall Grade: A+          ║
║   Standing   : OUTSTANDING / PASSED WITH DISTINCTION          ║
╚═══════════════════════════════════════════════════════════════╝
```

Chapter 17: Student Report Card

The **Student Report Card** generates an academic transcript displaying subject-wise scores, individual grades, aggregate marks, overall percentage, and a final `PASS` or `FAIL` verdict:

```
╔═══════════════════════════════════════════════════════════════════════╗
║                     ACADEMIC REPORT CARD                              ║
║                   SCHOOL MANAGEMENT SYSTEM                            ║
╠═══════════════════════════════════════════════════════════════════════╣
║ Student ID   : S101              Class : CS-101                       ║
║ Student Name : Alice Johnson     Age   : 19 years                     ║
╠═══════════════════════════════════════════════════════════════════════╣
║ SUBJECT NAME                │  OBTAINED │   TOTAL │   PERCENT │ GRADE ║
╠═══════════════════════════════════════════════════════════════════════╣
║ Programming                 │       95.0 │     100 │     95.0% │    A+ ║
║ Data Structures             │       91.0 │     100 │     91.0% │    A+ ║
╠═══════════════════════════════════════════════════════════════════════╣
║ Total Marks        : 186 / 200                                        ║
║ Overall Percentage : 93.00%                                           ║
║ Overall Grade      : A+                                               ║
║ Academic Status    : PASS                                             ║
╚═══════════════════════════════════════════════════════════════════════╝
```

Chapter 18: Class Merit List

The **Class Merit List** generates ranked leaderboards for individual cohorts or the entire student body, sorted in descending order of academic percentage.

18.1 Equal Percentage Tie-Handling

Standard competition ranking is used: students with identical percentages receive the same rank position, and the next rank skips accordingly:

```
+------+------------+--------------------------+------------+------------+-------+----+
| Rank | Student ID | Student Name             | Class      | Percentage | Grade | St |
+------+------------+--------------------------+------------+------------+-------+----+
| #1   | S101       | Alice Johnson            | CS-101     |     93.00% |    A+ |  P |
| #2   | S102       | Bob Smith                | CS-101     |     85.00% |     A |  P |
| #2   | S103       | Charlie Brown            | CS-101     |     85.00% |     A |  P |
+------+------------+--------------------------+------------+------------+-------+----+
Note: Students with equal percentage share the same merit position rank.
```

Chapter 19: File Handling and Data Storage

19.1 Flat File Architecture

Data is persistently stored in three plain text files located in the `data/` directory. A single delimiter character—the vertical bar or pipe (`'|'`)—separates individual record fields:

Database File	Field Schema Format	Live Example Record
`data/students.txt`	`id|name|age|studentClass`	`S101|Alice Johnson|19|CS-101`
`data/attendance.txt`	`studentId|date|status`	`S101|2026-11-01|Present`
`data/results.txt`	`studentId|subject|obtainedMarks|totalMarks`	`S101|Programming|95|100`

19.2 Reading Records via `std::ifstream`

When the application starts, each manager executes `loadFromFile()`:

```
bool StudentManager::loadFromFile() {
    students.clear();
    std::ifstream inFile(filePath);
    if (!inFile.is_open()) return true; // Safe if file does not exist yet

    std::string line;
    while (std::getline(inFile, line)) {
        line = UI::trim(line);
        if (line.empty()) continue;
        Student s = Student::fromFileString(line);
        if (!s.getId().empty()) {
            students.push_back(s);
        }
    }
    inFile.close();
    return true;
}
```

19.3 Writing Records via `std::ofstream` with Truncation

When data is inserted, updated, or deleted, the in-memory vector is modified and `saveToFile()` rewrites the entire file using `std::ios::trunc`. This ensures consistency between memory and disk storage without leaving stale records:

```
bool StudentManager::saveToFile() const {
    std::filesystem::path p(filePath);
    if (p.has_parent_path()) {
        std::filesystem::create_directories(p.parent_path());
    }

    std::ofstream outFile(filePath, std::ios::trunc);
    if (!outFile.is_open()) return false;

    for (const auto& s : students) {
        outFile << s.toFileString() << "
";
    }
    outFile.close();
    return true;
```

```
}
```

Chapter 20: Complete Program Execution Flow

The application follows a structured execution sequence:

1. **Platform Initialization:** `UI::initConsole()` enables UTF-8 and ANSI escape sequence processing in the Windows console.
2. **Manager Instantiation:** `StudentManager`, `AttendanceManager`, and `ExaminationManager` objects are instantiated in `main()`.
3. **Startup Deserialization:** Each manager reads its corresponding file in `data/`, populating its in-memory vector. Missing files are handled gracefully.
4. **Main Event Loop:** The console displays the Main Menu. The user selects a module via `UI::getIntInput()`.
5. **Sub-menu Dispatch:** User input is routed to the corresponding controller function:
   - `runStudentMenu()` for Student Information Management.
   - `runAttendanceMenu()` for Attendance Management.
   - `runExaminationMenu()` for Examination Results Management.
6. **Execution and Synchronized Persistence:** Sub-menu operations modify the in-memory vectors and immediately persist changes to disk.
7. **Program Termination:** Selecting `0` from the Main Menu exits the event loop, displays a farewell message, and returns `0` from `main()`.

Chapter 21: Algorithms and Calculations

21.1 Attendance Percentage Formula

Attendance percentage is computed using safe floating-point division with a zero-division guard:

```
if (totalDays == 0) return 0.0;
double percentage = (static_cast<double>(presentCount) / totalDays) * 100.0;
```

21.2 Academic Performance & Automated Grading

Subject percentage is calculated from obtained and total marks:

```
double percentage = (obtainedMarks / totalMarks) * 100.0;
```

The letter grade is determined using the institutional grading scale:

Percentage Range	Letter Grade	Status Interpretation
$90.0\% - 100.0\%$	**A+**	Outstanding / High Distinction
$80.0\% - 89.99\%$	**A**	Excellent / Distinction
$70.0\% - 79.99\%$	**B**	Good / Commendable
$60.0\% - 69.99\%$	**C**	Satisfactory / Average
$50.0\% - 59.99\%$	**D**	Pass / Minimum Competency
Below 50.0%	**F**	Fail / Unsatisfactory

21.3 Competition Ranking Algorithm

To rank students for the Class Merit List, summaries are sorted in descending order of percentage. Equal percentages are detected using an epsilon check ($|a - b| \le 0.0001$) and assigned the same rank:

```
std::sort(summaries.begin(), summaries.end(), [](const StudentAcademicSummary& a, const StudentAcademic
    if (std::abs(a.percentage - b.percentage) > 0.0001)
        return a.percentage > b.percentage;
    return a.totalObtained > b.totalObtained;
});

int currentRank = 1;
for (size_t i = 0; i < summaries.size(); ++i) {
    if (i > 0 && std::abs(summaries[i].percentage - summaries[i - 1].percentage) > 0.0001) {
        currentRank = static_cast<int>(i + 1);
    }
    summaries[i].rank = currentRank;
}
```

Chapter 22: Input Validation and Error Handling

22.1 Safe Console Stream Handling

Using `std::cin >> var` directly can cause terminal locks when non-numeric data is entered into a numeric variable. To prevent this, all console input in this project uses `std::getline(std::cin, line)` followed by strict validation and conversion:

```
size_t idx = 0;
int val = std::stoi(line, &idx);
if (idx != line.length()) {
    // Rejects inputs with trailing non-numeric characters like "15abc"
}
```

22.2 Delimiter Protection

Because fields are separated by `'|'` in the data files, any user input containing a pipe character could corrupt the storage format. The input reader explicitly checks for and rejects this character:

```
if (input.find('|') != std::string::npos) {
    printError("Character '|' is reserved. Please avoid using it.");
    continue;
}
```

22.3 Boundary Enforcement

- **Student Age:** Enforced in the range $[4, 100]$.
- **Exam Marks:** Total Marks must be ≥ 1.0. Obtained Marks must be ≥ 0.0 and $\le ext{Total Marks}$.
- **Calendar Dates:** Validated in strict `YYYY-MM-DD` format (e.g., Year 2000–2099, Month 1–12, Day 1–31).

Chapter 23: Compilation and Execution Instructions

23.1 Compilation via Command Line

Open PowerShell or Command Prompt, navigate to the project directory, and compile using GCC / MinGW:

```
cd "C:\Users\user\OneDrive\Desktop\School-Management-System"
g++ -std=c++17 -Wall -Wextra main.cpp student.cpp attendance.cpp examination.cpp -o SchoolManagementSys
```

23.2 Using the Provided Batch Scripts

- **`compile.bat`**: Executes the `g++` command and displays a success or error notification based on the exit code.
- **`run.bat`**: Verifies that `SchoolManagementSystem.exe` exists (triggering compilation if missing) and launches the application.

Chapter 24: Testing and Sample Outputs

24.1 Edge Case Test Scenarios

Test Scenario	Input Provided	System Response	Result
Duplicate Student ID	ID: "S101" (already registered)	"[ERROR] Student ID 'S101' already exists!"	PASS
Invalid Exam Marks	Obtained: 105, Total: 100	"[ERROR] Value must be between 0 and 100."	PASS
Duplicate Attendance	ID: "S101", Date: "2026-11-01" twice	"[ERROR] Attendance already recorded for date!"	PASS
Non-numeric Input	Typing "abc" at integer menu prompt	"[ERROR] Please enter a valid whole number."	PASS
Cascading Deletion	Delete student "S101"	Purges matching records from attendance.txt & results.txt	PASS

Chapter 25: Project Limitations

Documenting known system limitations is an essential component of academic software engineering:

- **In-Memory Working Set:** All records are loaded into memory on startup. While well-suited for several thousand students, datasets with hundreds of thousands of records would require chunked or indexed disk access.
- **File Rewrite Overhead:** File updates rewrite the entire dataset via `std::ios::trunc` ($O(N)$ write cost), rather than modifying records in-place.
- **Linear Search Complexity:** Student lookups use linear scans ($O(N)$) across vectors. Using a hash index (`std::unordered_map<std::string, Student>`) would reduce lookup time to $O(1)$.
- **Single-User Architecture:** The application runs in a single process and does not implement file locking or concurrent multi-user synchronization.

Chapter 26: Conclusion

The **School Management System** demonstrates how core computer science principles—object-oriented encapsulation, clean data structures, defensive input sanitization, and deterministic file handling—can be combined to build a robust administrative application in standard C++17.

By eschewing heavyweight external libraries in favor of the C++ Standard Template Library, the codebase remains portable, transparent, and easy to inspect. The resulting architecture serves as a solid foundation for further extensions, such as database integration or graphical user interfaces.

Chapter 27: University Viva Questions and Answers

This chapter contains **70 university viva examination questions and answers** (50 Basic & Intermediate, and 20 Advanced & Challenging) tailored specifically to the architecture, code, and design of this School Management System project.

Section 27.1: Basic and Intermediate Viva Questions (Q1 – Q50)

Q1: What is the main objective of this School Management System?
Answer: The primary objective is to manage three core school operations using standard C++17: Student Information, Student Attendance, and Examination Results, backed by persistent text files, without using external GUI or database frameworks.

Q2: Which C++ standard does your project use, and why?
Answer: It uses C++17. We use C++17 specifically for `<filesystem>` (to create directories automatically), lambda expressions in algorithms, and modern standard library features.

Q3: Why did you not use a database like MySQL or SQLite?
Answer: The project requirement was to build a zero-dependency, self-contained educational system using standard C++ file handling (`fstream`) to demonstrate file streams, serialization, and parsing at a fundamental level.

Q4: How many main modules are in the application?
Answer: There are three main modules: Module 1 for Student Information Management, Module 2 for Attendance Management, and Module 3 for Examination Results Management, plus an Exit option.

Q5: What are the three extra features integrated into these modules?
Answer: 1) The Student Profile Card (an all-in-one console dashboard in Module 1), 2) The Student Report Card (a detailed academic transcript in Module 3), and 3) The Class Merit List (a ranked leaderboard in Module 3).

Q6: What is a header file (.h), and what is its purpose?
Answer: A header file contains class declarations, function prototypes, constants, and type definitions. It defines the public interface of a module so other translation units can interact with it without needing to know its implementation details.

Q7: What is the purpose of header guards like `#ifndef STUDENT_H`?
Answer: Header guards prevent double inclusion. If a header is included multiple times across different files, the preprocessor ensures its contents are compiled only once, avoiding redefinition errors.

Q8: What is encapsulation, and where is it used in your code?
Answer: Encapsulation is bundling data and the methods that operate on it into a class while restricting direct access to the data. In `Student`, attributes like `id`, `name`, `age`, and `studentClass` are `private`, and can only be accessed through public getters and setters.

Q9: What is the difference between `private` and `public` access specifiers?
Answer: `private` members can only be accessed by member functions of the same class. `public` members can

be called from outside the class, such as from `main()` or manager classes.

Q10: What is a constructor? How many constructors does your `Student` class have?
Answer: A constructor is a special member function invoked automatically when an object is instantiated. `Student` has two constructors: a default constructor initializing empty values, and a parameterized constructor initializing ID, name, age, and class.

Q11: Why is a default constructor necessary in entity classes?
Answer: When reading records from files or declaring uninitialized objects, C++ requires a default constructor to create a blank instance before populating its fields.

Q12: What is the purpose of the `UI` namespace?
Answer: The `UI` namespace groups ANSI color constants, Windows console setup, and safe input functions. Using a namespace prevents global name pollution and avoids name collisions with the standard library.

Q13: How do ANSI escape sequences work for console colors?
Answer: ANSI escape codes are special character sequences beginning with `\033[` followed by code numbers and `m`. When sent to the terminal, the terminal driver intercepts them and changes the text color, weight, or background.

Q14: How does `UI::initConsole()` enable ANSI colors on Windows?
Answer: It retrieves the console output handle using `GetStdHandle(STD_OUTPUT_HANDLE)`, reads the console mode via `GetConsoleMode`, and enables the `ENABLE_VIRTUAL_TERMINAL_PROCESSING` flag using bitwise-OR.

Q15: Why did you use `SetConsoleOutputCP(CP_UTF8)`?
Answer: Windows consoles default to OEM code pages like CP437. Switching to UTF-8 (Code Page 65001) allows clean rendering of Unicode box characters (such as ╔, ═, and ╚).

Q16: Why did you use `std::vector` instead of raw arrays?
Answer: `std::vector` is a dynamic array from the STL that handles its own memory allocation, automatically grows as records are added, and provides utility methods like `size()`, `push_back()`, and iterator support.

Q17: What is the purpose of `std::stringstream` in your project?
Answer: `std::stringstream` treats an in-memory `std::string` as a stream, allowing us to parse delimited fields using `std::getline(ss, token, '|')`.

Q18: What is the delimiter used in your data files, and why?
Answer: We use the pipe character (`'|'`). Commas frequently appear in names, and spaces appear in full names and subject titles, whereas pipe characters are rarely used in standard text.

Q19: How do you prevent users from breaking the file format?
Answer: In `UI::getStringInput()`, any input containing `'|'` is rejected with an error message, preventing delimiter collision.

Q20: What problem occurs if you use `std::cin >> variable` for numbers?
Answer: If the user enters letters instead of numbers, `std::cin` enters a fail state, sets error bits, and stops processing input, which can send the program into an infinite loop.

Q21: How does your code prevent this input crash?
Answer: In `UI::getIntInput()` and `UI::getDoubleInput()`, we always read the input as a string using `std::getline`, then convert it using `std::stoi` or `std::stod` inside a `try-catch` block.

Q22: What does `std::stoi(line, &idx)` do with the second argument `&idx`?
Answer: `idx` stores the character index where numeric parsing stopped. If `idx != line.length()`, it indicates the user entered non-numeric trailing characters (like "25abc"), allowing us to reject the input.

Q23: How are duplicate Student IDs prevented?
Answer: Before adding a student, `StudentManager::handleAddStudent()` calls `studentExists(id)`. If a match is found, an error is displayed and the insertion is aborted.

Q24: How does `StudentManager::findStudent()` indicate that a student was not found?
Answer: It returns a pointer (`Student*` or `const Student*`). If the student is found, it returns the memory address of the object; if not found, it returns `nullptr`.

Q25: Why is `findStudent()` overloaded with both `const` and non-`const` versions?
Answer: The `const` version allows inspection without modifying the student, while the non-`const` version allows mutating methods (like `updateStudent()`) to modify student fields directly.

Q26: What happens when a student record is updated?
Answer: The user enters new values or presses Enter to keep existing ones. The object in the `students` vector is modified, and `saveToFile()` rewrites `students.txt` to persist the changes.

Q27: How does student deletion work?
Answer: It uses the C++ erase-remove idiom with `std::remove_if` and a lambda function matching the ID, erases the element from the vector, and calls `saveToFile()`.

Q28: What is cascading deletion synchronization?
Answer: When a student is deleted from `StudentManager`, `main.cpp` automatically calls `attendanceMgr.deleteRecordsForStudent(id)` and `examMgr.deleteRecordsForStudent(id)`, removing all related records to prevent orphan data.

Q29: How does the system prevent duplicate attendance records?
Answer: In `AttendanceManager::hasAttendance(studentId, date)`, it scans existing records. If the student already has an attendance log for that calendar date, the entry is rejected.

Q30: How is attendance percentage calculated?
Answer: The system counts the number of "Present" records for a student and divides by total recorded sessions: $(ext{Present Days} / ext{Total Sessions}) imes 100$. If total sessions is zero, it returns 0.0 to prevent division by zero.

Q31: What attendance threshold triggers a warning?
Answer: If attendance falls below 75%, the system displays a warning message and flags the student's status as having an attendance shortage.

Q32: What validations are performed when entering examination marks?
Answer: Total Marks must be ≥ 1.0, and Obtained Marks must be ≥ 0.0 and $\le ext{Total Marks}$. It also checks whether the student exists and whether marks for that subject were already entered.

Q33: What happens if marks for a subject are entered a second time?
Answer: The system warns that a record already exists for that subject and asks the user whether they want to overwrite the marks. If confirmed, it updates the existing entry instead of adding a duplicate.

Q34: What is the grading scale used in the project?
Answer: $\ge 90\%$: A+, $80-89\%$: A, $70-79\%$: B, $60-69\%$: C, $50-59\%$: D, and below 50%: F.

Q35: What criteria determines whether a student passes or fails on the Report Card?
Answer: A student passes if their aggregate percentage is $\ge 50.0\%$ AND they have no failing grade (F) in any individual subject.

Q36: What is the purpose of the `StudentAcademicSummary` struct?
Answer: It serves as a Data Transfer Object (DTO) that holds aggregated statistics—total marks, total obtained, percentage, grade, fail status, and rank—making report and merit calculations clean and reusable.

Q37: What is the difference between a `struct` and a `class` in C++?
Answer: In C++, the only technical difference is default access: members of a `struct` default to `public`, whereas members of a `class` default to `private`.

Q38: How does the Class Merit List handle students with equal percentages?
Answer: It uses standard competition ranking. Students with equal percentages share the same rank position (e.g., two students tied at Rank #2), and the next rank skips to #4.

Q39: Why is floating-point comparison with `std::abs() < 0.0001` used for ranks?
Answer: Floating-point numbers can have small precision inaccuracies due to binary representation. Comparing whether $|a - b| \le 0.0001$ ensures safe equality checks for percentages.

Q40: What happens if a data file does not exist when the program runs for the first time?
Answer: In `loadFromFile()`, if `!inFile.is_open()`, it returns `true` cleanly. When records are later added, `saveToFile()` creates the directory and file automatically.

Q41: What does `std::ios::trunc` do in `std::ofstream`?
Answer: `std::ios::trunc` truncates (clears) the file's contents before writing, ensuring that rewritten vectors replace old data cleanly without leaving trailing characters.

Q42: What does `std::setw()` from `<iomanip>` do?
Answer: `std::setw(n)` sets the minimum field width for the next output item to n characters, enabling aligned columns in console tables.

Q43: What is the purpose of `std::left` and `std::right`?
Answer: They control text justification within a field formatted with `std::setw()`. `std::left` left-aligns text, while `std::right` right-aligns numbers.

Q44: What is a lambda function in C++, and where did you use one?
Answer: A lambda is an anonymous inline function. We used lambdas as predicates in `std::remove_if` to find records matching a given Student ID, and in `std::sort` to order merit summaries.

Q45: What does `const` after a member function declaration mean?
Answer: It signifies a const member function, which guarantees that calling it will not modify any member variables of the calling object.

Q46: Why is pass-by-reference (`const Type&`) preferred over pass-by-value?
Answer: Pass-by-value copies the entire object into function memory. Pass-by-reference passes only the memory address, eliminating copy overhead while `const` protects the original data from modification.

Q47: What does the compiler flag `-Wall` mean?
Answer: `-Wall` tells the GCC compiler to enable "all standard warnings," alerting the developer to uninitialized variables, unused parameters, or type mismatches.

Q48: What does `-Wextra` do?
Answer: `-Wextra` enables additional compiler checks beyond `-Wall`, catching subtle issues like unsigned/signed integer comparisons and unused return values.

Q49: How does the application ensure clean exit?
Answer: Selecting `0` in the main menu breaks the main event loop, flushes any output buffers, displays a farewell message, and returns `0` from `main()`.

Q50: Can this system run on Linux or macOS?
Answer: Yes. The core logic uses standard C++17. The Windows-specific headers in `student.h` are guarded by `#ifdef _WIN32`, so on Linux or macOS, the code compiles using standard POSIX ANSI terminal output.

Section 27.2: Advanced and Challenging Viva Questions (Q51 – Q70)

Q51: Why did your design use three separate manager classes instead of a single giant Manager class?
Answer: This follows the Single Responsibility Principle (SRP) from SOLID design. Each manager is responsible for exactly one domain (students, attendance, or examinations). Combining them into one God class creates high coupling and makes maintenance difficult.

Q52: Why did you not use C++ inheritance between Student and Manager classes?
Answer: Inheritance represents an "is-a" relationship (e.g., a GraduateStudent is a Student). A Manager is not a Student; it manages a collection of students. Applying inheritance here would violate object-oriented principles.

Q53: How does the erase-remove idiom work in `deleteStudent()`?
Answer: `std::remove_if` moves elements that do NOT match the predicate to the front of the vector and returns an iterator to the new logical end. `vector::erase` then deallocates the remaining elements from that iterator to `end()`.

Q54: What is the time complexity of searching for a student by ID in your system?
Answer: It is $O(N)$, because `findStudent()` performs a linear scan through the `std::vector`. In a production

system with millions of records, an `std::unordered_map` could be used for $O(1)$ average lookup time.

Q55: What is the time complexity of the Class Merit List sorting algorithm?
Answer: `std::sort` uses IntroSort (a hybrid of QuickSort, HeapSort, and InsertionSort), which provides an optimal worst-case time complexity of $O(N \log N)$.

Q56: What happens if two students have identical names, ages, and classes?
Answer: The system distinguishes them by their unique Student ID (primary key). As long as their IDs differ (e.g., S101 and S102), they are stored and managed as distinct entities.

Q57: Why is it important to use `std::filesystem::create_directories` before opening an `ofstream`?
Answer: If an `std::ofstream` attempts to open a file in a subfolder (like `data/students.txt`) and the directory does not exist, the stream fails silently and no file is created. `create_directories` guarantees the path exists first.

Q58: Why does `std::setw()` misalign if ANSI escape codes are placed inside the formatted string?
Answer: `std::setw()` calculates padding based on the raw byte length of the string. ANSI escape sequences contain invisible bytes (like `[93m`), which count toward the byte length and cause visible column misalignment.

Q59: How did you solve the ANSI misalignment problem in the Report Card and Profile Card?
Answer: We used custom row printers (`printCardRow` and `printReportRow`) that compute padding based strictly on visible character length, keeping the border characters (‖) aligned.

Q60: What happens if a user enters non-English Unicode characters in a student's name?
Answer: Because the console is initialized to UTF-8 via `SetConsoleOutputCP(CP_UTF8)` and inputs are handled as dynamic `std::string` byte streams, valid UTF-8 sequences are preserved and stored correctly.

Q61: What would happen if power failed while `saveToFile()` was executing?
Answer: Because `saveToFile()` uses `std::ios::trunc`, an abrupt power failure during the write could leave a partially written file. A production solution would use atomic writes (writing to a `.tmp` file first, then replacing the target via `std::filesystem::rename`).

Q62: Why did you not use virtual functions or polymorphism in this project?
Answer: The project uses homogeneous collections—every element in `students` is a concrete `Student` instance. Polymorphism is beneficial when dealing with class hierarchies, but adding virtual tables here would introduce unnecessary indirection without practical benefit.

Q63: What design pattern does your data access most closely resemble?
Answer: It closely resembles the **Repository Pattern**. Each manager abstracts storage operations behind domain-specific methods (`addStudent`, `findStudent`, `deleteStudent`), isolating data access from presentation logic.

Q64: How does `ExaminationManager` interact with `StudentManager` without circular dependency?
Answer: `ExaminationManager` does not own or store `StudentManager`. Instead, methods that require student verification (like `handleEnterMarks`) accept a `const StudentManager&` as a function parameter, keeping coupling loose and directional.

Q65: How do you handle empty lines or corrupted rows in the text files?
Answer: In `fromFileString()`, parsing is enclosed in a `try-catch` block. If conversion fails, it returns an empty object with an empty ID. `loadFromFile()` checks `!s.getId().empty()` before adding the record to the vector.

Q66: Why did you use member initializer lists in your constructors?
Answer: Member initializer lists directly initialize attributes in place during memory allocation, avoiding the overhead of default construction followed by an assignment operator call.

Q67: How does your system ensure date validation for leap years?
Answer: The current `getDateInput()` verifies general calendar boundaries (Month 1-12, Day 1-31). A strict production enhancement could add a leap-year rule ($(ext{year} \% 4 == 0 \ \&\&\ ext{year} \% 100 e 0) \ ||\ (ext{year} \% 400 == 0)$) for February.

Q68: What is the benefit of keeping data storage in plain text over binary serialization?
Answer: Plain text is human-readable, portable across different operating systems and compiler architectures, and easy to inspect and debug directly in text editors without custom deserialization tools.

Q69: If this system were expanded to 100,000 students, what architectural changes would you make?
Answer: 1) Replace linear vector searches with indexed hash maps (`std::unordered_map`). 2) Switch from flat-file rewrites to an embedded ACID database like SQLite or B-Tree indexing. 3) Implement pagination in console display tables.

Q70: How would you summarize the core strength of your implementation?
Answer: The project balances clean object-oriented architecture with defensive input sanitization, complete data persistence, cascading referential integrity, and an aligned, color-coded console interface using only standard ISO C++17.